

```

1 // Comments marked by initials ER, KP, KL, DS, EC
2 //#####
3 // FILE:    FinalProjectStarter_main.c
4 //
5 // TITLE:   Final Project Starter
6 //#####
7
8 // Included Files
9 #include <stdio.h>
10 #include <stdlib.h>
11 #include <stdarg.h>
12 #include <string.h>
13 #include <math.h>
14 #include <limits.h>
15 #include "F28x_Project.h"
16 #include "F2837xD_SWPrioritizedIsrLevels.h"
17 #include "driverLib.h"
18 #include "device.h"
19 #include "F28379dSerial.h"
20 #include "song.h"
21 #include "dsp.h"
22 #include "fpu32/fpu_rfft.h"
23 #include "xy.h"
24 #include "MatrixMath.h"
25 #include "SE423Lib.h"
26 #include "OptiTrack.h"
27
28 #define PI          3.1415926535897932384626433832795
29 #define TWOPI       6.283185307179586476925286766559
30 #define HALFPI      1.5707963267948966192313216916398
31 // The Launchpad's CPU Frequency set to 200 you should not change this value
32 #define LAUNCHPAD_CPU_FREQUENCY 200
33
34 // ----- code for CAN start here -----
35 #include "F28379dCAN.h"
36 // #define TX_MSG_DATA_LENGTH      4
37 // #define TX_MSG_OBJ_ID           0 //transmit
38
39 #define RX_MSG_DATA_LENGTH      8
40 #define RX_MSG_OBJ_ID_1        1 //measurement from sensor 1
41 #define RX_MSG_OBJ_ID_2        2 //measurement from sensor 2
42 #define RX_MSG_OBJ_ID_3        3 //quality from sensor 1
43 #define RX_MSG_OBJ_ID_4        4 //quality from sensor 2
44 // ----- code for CAN end here -----
45
46 // Interrupt Service Routines predefinition
47 __interrupt void cpu_timer0_isr(void);
48 __interrupt void cpu_timer1_isr(void);
49 __interrupt void cpu_timer2_isr(void);
50 __interrupt void SWI1_HighestPriority(void);
51 __interrupt void SWI2_MiddlePriority(void);
52 __interrupt void SWI3_LowestPriority(void);
53 __interrupt void ADCC_ISR(void);
54 __interrupt void SPIB_isr(void);
55 // ----- code for CAN start here -----
56 __interrupt void can_isr(void);
57 // ----- code for CAN end here -----
58
59 void setF28027EPWM1A(float controleffort);
60 int16_t EPwm1A_F28027 = 1500;
61 void setF28027EPWM2A(float controleffort);
62 int16_t EPwm2A_F28027 = 1500;
63
64 // ----- code for CAN start here -----
65 // volatile uint32_t txMsgCount = 0;
66 // extern uint16_t txMsgData[4];
67
68 volatile uint32_t rxMsgCount_1 = 0;
69 volatile uint32_t rxMsgCount_2 = 0;
70 extern uint16_t rxMsgData[8];
71
72 uint32_t dis_raw_1[2];
73 uint32_t dis_raw_2[2];
74 uint32_t dis_1 = 0;
75 uint32_t dis_2 = 0;
76
77 uint32_t quality_raw_1[4];
78 uint32_t quality_raw_2[4];
79 float quality_1 = 0.0;
80 float quality_2 = 0.0;
81
82 uint32_t lightlevel_raw_1[4];
83 uint32_t lightlevel_raw_2[4];
84 float lightlevel_1 = 0.0;
85 float lightlevel_2 = 0.0;
86
87 uint32_t measure_status_1 = 0;
88 uint32_t measure_status_2 = 0;
89
90 volatile uint32_t errorFlag = 0;
91 // ----- code for CAN end here -----
92
93 uint32_t numTimer0calls = 0;
94 uint16_t UARTPrint = 0;
95
96 float printLV1 = 0;
97 float printLV2 = 0;
98
99 float printLinux1 = 0;
100 float printLinux2 = 0;
101
102 uint16_t LADARi = 0;
103 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
104 uint16_t G_len = 11; //length of command
105 xy ladar_pts[228]; //xy data
106 float LADARrightfront = 0;
107 float LADARfront = 0;
108 float LADARtemp_x = 0;
109 float LADARtemp_y = 0;
110 extern datapts ladar_data[228];
111
112 extern uint16_t newLinuxCommands;
113 extern float LinuxCommands[CMDNUM_FROM_FLOATS];
114
115 extern uint16_t NewLVData;
116 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
117 extern LVSendFloats_t DataToLabView;
118 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
119 extern uint16_t LADARpingpong;
120 extern uint16_t NewCAMDataThreshold1; // Flag new data
121 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
122 extern uint16_t NewCAMDataThreshold2; // Flag new data
123 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
124
125 extern uint16_t NewCAMDataAprilTag1; // Flag new data
126 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
127
128 float tagid = 0;
129 float tagx = 0;
130 float tagy = 0;
131 float tagz = 0;
132 float tagthetax = 0;
133 float tagthetay = 0;
134 float tagthetaz = 0;
135 int32_t numtag = 0;
136 int32_t AprilTagState = 10;
137 float KpAprilTag = -1.0;
138 float AprilTagSetPoint = 0;
139 int32_t AprilTagCount = 0;
140

```

```

141 float MaxAreaThreshold1 = 0;
142 float MaxColThreshold1 = 0;
143 float MaxRowThreshold1 = 0;
144 float NextLargestAreaThreshold1 = 0;
145 float NextLargestColThreshold1 = 0;
146 float NextLargestRowThreshold1 = 0;
147 float NextNextLargestAreaThreshold1 = 0;
148 float NextNextLargestColThreshold1 = 0;
149 float NextNextLargestRowThreshold1 = 0;
150
151 float MaxAreaThreshold2 = 0;
152 float MaxColThreshold2 = 0;
153 float MaxRowThreshold2 = 0;
154 float NextLargestAreaThreshold2 = 0;
155 float NextLargestColThreshold2 = 0;
156 float NextLargestRowThreshold2 = 0;
157 float NextNextLargestAreaThreshold2 = 0;
158 float NextNextLargestColThreshold2 = 0;
159 float NextNextLargestRowThreshold2 = 0;
160
161 uint32_t numThres1 = 0;
162 uint32_t numThres2 = 0;
163
164 pose ROBOTps = {0,0,0}; //robot position
165 pose LADARps = {3.5/12,0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
166 float LADARxoffset = 0;
167 float LADARyoffset = 0;
168
169 uint32_t timecount = 0;
170 int16_t RobotState = 1;
171 int16_t checkfrontally = 0;
172 int32_t WallFollowtime = 0;
173
174 #define NUMWAYPOINTS 10 //Modified NUMWAYPOINTS from 8 to 10 for two extra waypoint targets for the robot path - DS
175 uint16_t statePos = 0;
176 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
177 uint16_t i = 0;//for loop
178
179 uint16_t right_wall_follow_state = 2; // right follow
180 float Kp_front_wall = -2.0;
181 float front_turn_velocity = 0.2;
182 float left_turn_Stop_threshold = 3.5;
183 float Kp_right_wal = -4.0;
184 float ref_right_wall = 1.1;
185 float foward_velocity = 1.0;
186 float left_turn_Start_threshold = 1.3;
187 float turn_saturation = 2.5;
188
189 //RC Servo
190 float Gate_0 = -67.74;
191 float Gate_C = 0.40;
192 float RTong = -49.34; //KLEC: ball goes to the left
193 float MTong = -11.49;
194 float LTong = 1.47; //KLEC: ball goes to the right
195
196 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
197
198 //more kalman vars
199 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
200 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
201 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
202 float z[3][1]; // state measurement
203 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
204 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
205 #define ProcUncert 0.0001
206 #define CovScalar 10
207 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
208 {0,ProcUncert,ProcUncert/CovScalar},
209 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
210 #define MeasUncert 1
211 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
212 {0,MeasUncert,MeasUncert/CovScalar},
213 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of OptiTrack Motion Capture measurement)
214 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
215 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
216 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
217 float temp_3x3[3][3]; // intermediate storage
218 float temp_3x1[3][1]; // intermediate storage
219 float ytilde[3][1]; // difference between predictions
220
221 // Optitrack Variables
222 int32_t OptiNumUpdatesProcessed = 0;
223 pose OPTITRACKps;
224 extern uint16_t new_optitrack;
225 extern float Optitrackdata[OPTITRACKDATASIZE];
226 int16_t newOPTITRACKpose=0;
227
228 int16_t adcc2result = 0;
229 int16_t adcc3result = 0;
230 int16_t adcc4result = 0;
231 int16_t adcc5result = 0;
232 float adcC2Volt = 0.0;
233 float adcC3Volt = 0.0;
234 float adcC4Volt = 0.0;
235 float adcC5Volt = 0.0;
236 int32_t numADCCcalls = 0;
237 float LeftWheel = 0;
238 float RightWheel = 0;
239 float LeftWheel_1 = 0;
240 float RightWheel_1 = 0;
241 float LeftVel = 0;
242 float RightVel = 0;
243 float uLeft = 0;
244 float uRight = 0;
245 float HandValue = 0;
246 float vref = 0;
247 float turn = 0;
248 float gyro9250_drift = 0;
249 float gyro9250_angle = 0;
250 float old_gyro9250 = 0;
251 float gyroLPRS10_angle = 0;
252 float gyroLPRS10_offset = 0;
253 float gyroLPRS10_drift = 0;
254 float old_gyroLPRS10 = 0;
255 float gyro9250_radians = 0;
256 float gyroLPRS10_radians = 0;
257
258 int16_t readdata[25];
259 int16_t IMU_data[9];
260
261 float accelx = 0;
262 float accely = 0;
263 float accelz = 0;
264 float gyrox = 0;
265 float gyroy = 0;
266 float gyroz = 0;
267
268 // Needed global Variables
269 float accelx_offset = 0;
270 float accely_offset = 0;
271 float accelz_offset = 0;
272 float gyrox_offset = 0;
273 float gyroy_offset = 0;
274 float gyroz_offset = 0;
275 int16_t calibration_state = 0;
276 int32_t calibration_count = 0;
277 int16_t doneCal = 0;
278
279 #define MPU9250 1
280 #define DAN28027 2
281 int16_t CurrentChip = MPU9250;
282 int16_t DAN28027Garbage = 0;

```

```

283 int16_t dan28027adc1 = 0;
284 int16_t dan28027adc2 = 0;
285 uint16_t MPUS250IgnoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
286 //Added variables to store camera-based ball distances - DS
287 float robotToBall2 = 0;
288 float robotToBall1 = 0;
289
290 // angle of the hand encoder - ER
291 float RCangle = 0.0;
292 float colcentroid1 = 0.0;
293 float colcentroid2 = 0.0;
294 float kpvision = -0.07; // vision proportional gain chosen so robot turns toward the ball centroid without overcorrecting - DS
295
296 int16_t count = 0;
297 int16_t statecount = 2000;
298
299 void main(void)
300 {
301     // PLL, WatchDog, enable Peripheral Clocks
302     // This example function is found in the F2837xD_SysCtrl.c file.
303     InitSysCtrl();
304
305     InitGpio();
306
307     InitSE423DefaultGPIO();
308
309     //Scope for Timing
310     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
311     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSH_PULL);
312     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
313
314     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
315     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSH_PULL);
316     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
317
318     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
319     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSH_PULL);
320     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
321
322     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
323     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSH_PULL);
324     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
325
326     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
327     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSH_PULL);
328     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
329
330     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
331     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSH_PULL);
332     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
333
334     // ----- code for CAN start here -----
335     //GPIO17 - CANRXB
336     GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
337     GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
338
339     //GPIO12 - CANTXB
340     GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
341     GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSH_PULL);
342     // ----- code for CAN end here -----
343
344
345
346     // ----- code for CAN start here -----
347     // Initialize the CAN controller
348     InitCANB();
349
350     // Set up the CAN bus bit rate to 1000 kbps
351     setCANBitRate(200000000, 1000000);
352
353     // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
354     CanbRegs.CAN_CTL.bit.IE0 = 1;
355     CanbRegs.CAN_CTL.bit.EIE = 1;
356     // ----- code for CAN end here -----
357
358     // Clear all interrupts and initialize PIE vector table:
359     // Disable CPU interrupts
360     DINT;
361
362     // Initialize the PIE control registers to their default state.
363     // The default state is all PIE interrupts disabled and flags
364     // are cleared.
365     // This function is found in the F2837xD_PieCtrl.c file.
366     InitPieCtrl();
367
368     // Disable CPU interrupts and clear all CPU interrupt flags:
369     IER = 0x0000;
370     IFR = 0x0000;
371
372     // Initialize the PIE vector table with pointers to the shell Interrupt
373     // Service Routines (ISR).
374     // This will populate the entire table, even if the interrupt
375     // is not used in this example. This is useful for debug purposes.
376     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
377     // This function is found in F2837xD_PieVect.c.
378     InitPieVectTable();
379
380     // Interrupts that are used in this example are re-mapped to
381     // ISR functions found within this project
382     // EALLOW; // This is needed to write to EALLOW protected registers
383     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
384     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
385     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
386     PieVectTable.SCIA_RX_INT = &RXAINT_rcv_ready;
387     PieVectTable.SCIB_RX_INT = &RXBINT_rcv_ready;
388     PieVectTable.SCIC_RX_INT = &RXCINT_rcv_ready;
389     PieVectTable.SCID_RX_INT = &RXDINT_rcv_ready;
390     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
391     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
392     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
393     PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
394     PieVectTable.ADCC1_INT = &ADCC_ISR;
395     PieVectTable.SPIB_RX_INT = &SPIB_isr;
396     PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
397     PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
398     PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
399     // ----- code for CAN start here -----
400     PieVectTable.CANB0_INT = &can_isr;
401     // ----- code for CAN end here -----
402     EDIS; // This is needed to disable write to EALLOW protected registers
403
404     // Initialize the CpuTimers Device Peripheral. This function can be
405     // found in F2837xD_CpuTimers.c
406     InitCpuTimers();
407
408     // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
409     // 200MHz CPU Freq, Period (in uSeconds)
410     ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
411     ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!! Important, Used to command LADAR every 100ms. Do not Change.
412     ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
413
414     // Enable CpuTimer Interrupt bit TIE
415     CpuTimer0Regs.TCR.all = 0x4000;
416     CpuTimer1Regs.TCR.all = 0x4000;
417     CpuTimer2Regs.TCR.all = 0x4000;
418
419     DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
420
421     init_serialSCIB(&SerialB, 19200);
422     init_serialSCIC(&SerialC, 19200);
423
424     for (LADARi = 0; LADARi < 228; LADARi++) {

```

```

425     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
426 }
427
428 init_e0EPs();
429 init_EPWMland2();
430 init_RCServoPWM_3AB_5AB_6A();
431 init_ADCsAndDACs();
432 setupSpib();
433
434 EALLOW;
435 EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
436 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
437 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
438 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
439 EPwm4Regs.TBCTR = 0x0; // Clear counter
440 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
441 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
442 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
443 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
444 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
445 EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
446 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
447 EDIS;
448
449 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
450 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
451 // to CPU-Timer 2: int 12 is for the SWI.
452 IER |= M_INT1;
453 IER |= M_INT6;
454 IER |= M_INT8; // SCIC SCID
455 IER |= M_INT9; // SCIA
456 IER |= M_INT12;
457 IER |= M_INT13;
458 IER |= M_INT14;
459
460 // Enable TINT0 in the PIE: Group 1 interrupt 7
461 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
462 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
463 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
464 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
465 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
466 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
467 // ----- code for CAN start here -----
468 // Enable CANB in the PIE: Group 9 interrupt 7
469 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
470 // ----- code for CAN end here -----
471
472 // ----- code for CAN start here -----
473 // Enable the CAN interrupt signal
474 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
475 // ----- code for CAN end here -----
476
477
478 robotdest[0].x = -4;    robotdest[0].y = 10;
479 robotdest[1].x = -4;    robotdest[1].y = 2;
480 //middle of bottom
481 robotdest[2].x = 0;     robotdest[2].y = 2;
482 //outside the course
483 robotdest[3].x = 0;     robotdest[3].y = -3;
484 //back to middle
485 robotdest[4].x = 0;     robotdest[4].y = 2;
486 robotdest[5].x = 4;     robotdest[5].y = 2;
487 robotdest[6].x = 4;     robotdest[6].y = 10;
488 robotdest[7].x = 0;     robotdest[7].y = 9;
489 // the two extra points that we added for the robot to go to after the initial course one in the middle of the field and one outside near the wall - ER
490 robotdest[8].x = 2;     robotdest[8].y = 5;
491 robotdest[9].x = -2;    robotdest[9].y = -1;
492
493 // ROBOTps will be updated by Optitrack during gyro calibration
494 // TODO: specify the starting position of the robot
495 ROBOTps.x = 0;          //the estimate in array form (useful for matrix operations)
496 ROBOTps.y = 0;
497 ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
498 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
499 x_pred[1][0] = ROBOTps.y;
500 x_pred[2][0] = ROBOTps.theta;
501
502 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
503 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
504 SpibRegs.SPIFFRX.bit.RXFFOVFLCLR=1; // Clear Overflow flag
505 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
506 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
507 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
508
509 init_serialSCIA(&SerialA,115200);
510 init_serialSCID(&SerialD,2083332);
511
512 // Enable global Interrupts and higher priority real-time debug events
513 EINT; // Enable Global interrupt INTM
514 ERTM; // Enable Global realtime interrupt DBGM
515 // ----- code for CAN start here -----
516 // Measured Distance from 1
517 // Initialize the receive message object 1 used for receiving CAN messages.
518 // Message Object Parameters:
519 //     Message Object ID Number: 1
520 //     Message Identifier: 0x060b0101
521 //     Message Frame: Standard
522 //     Message Type: Receive
523 //     Message ID Mask: 0x0
524 //     Message Object Flags: Receive Interrupt
525 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
526 //     for a Receive mailbox)
527 //
528 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
529 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
530 RX_MSG_DATA_LENGTH);
531
532 // Measured Distance from 2
533 // Initialize the receive message object 2 used for receiving CAN messages.
534 // Message Object Parameters:
535 //     Message Object ID Number: 2
536 //     Message Identifier: 0x060b0102
537 //     Message Frame: Standard
538 //     Message Type: Receive
539 //     Message ID Mask: 0x0
540 //     Message Object Flags: Receive Interrupt
541 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
542 //     for a Receive mailbox)
543 //
544 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
545 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
546 RX_MSG_DATA_LENGTH);
547
548 // Measurement Quality from 1
549 // Initialize the receive message object 2 used for receiving CAN messages.
550 // Message Object Parameters:
551 //     Message Object ID Number: 3
552 //     Message Identifier: 0x060b0201
553 //     Message Frame: Standard
554 //     Message Type: Receive
555 //     Message ID Mask: 0x0
556 //     Message Object Flags: Receive Interrupt
557 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
558 //     for a Receive mailbox)
559 //
560 //
561 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
562 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
563 RX_MSG_DATA_LENGTH);
564
565 // Measurement Quality from 2

```

```

567 // Initialize the receive message object 2 used for receiving CAN messages.
568 // Message Object Parameters:
569 //     Message Object ID Number: 4
570 //     Message Identifier: 0x060b0202
571 //     Message Frame: Standard
572 //     Message Type: Receive
573 //     Message ID Mask: 0x0
574 //     Message Object Flags: Receive Interrupt
575 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
576 //     for a Receive mailbox)
577 //
578
579 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
580                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
581                       RX_MSG_DATA_LENGTH);
582
583 //
584 // Start CAN module operations
585 //
586 CanbRegs.CAN_CTL.bit.Init = 0;
587 CanbRegs.CAN_CTL.bit.CCE = 0;
588
589
590 // ----- code for CAN end here -----
591 char S_command[19] = "S1152000124000\n"; // this change the baud rate to 115200
592 uint16_t S_len = 19;
593 serial_sendSCIC(&SerialC, S_command, S_len);
594
595
596 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
597 init_serialSCIC(&SerialC, 115200);
598
599
600 // LED1 is GPIO22
601 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
602 // LED2 is GPIO94
603 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
604 // LED3 is GPIO95
605 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
606 // LED4 is GPIO97
607 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
608 // LED5 is GPIO111
609 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
610
611
612 // IDLE loop. Just sit and loop forever (optional):
613 while(1)
614 {
615     if (UARTPrint == 1) {
616         //UART_printfLine(1, "RCangle: %.2f", RCangle);
617         if (readbuttons() == 0) {
618             UART_printfLine(1, "RobotState: %d", RobotState);
619             UART_printfLine(1, "01A: %.0fC: %.0fR: %.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
620             UART_printfLine(2, "01A: %.0fC: %.0fR: %.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
621             // UART_printfLine(1, "x: %.2f; y: %.2f; a: %.2f", ROBOTps.x, ROBOTps.y, ROBOTps.theta);
622             UART_printfLine(2, "orange: %.2f", robotToBall2);
623         } else if (readbuttons() == 1) {
624             // UART_printfLine(1, "01A: %.0fC: %.0fR: %.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
625             // UART_printfLine(2, "01A: %.0fC: %.0fR: %.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
626             //UART_printfLine(1, "LV1: %.3f LV2: %.3f", printLV1, printLV2);
627             //UART_printfLine(2, "Ln1: %.3f Ln2: %.3f", printLinux1, printLinux2);
628         } else if (readbuttons() == 2) {
629             UART_printfLine(1, "02A: %.0fC: %.0fR: %.0f", NextLargestAreaThreshold1, NextLargestColThreshold1, NextLargestRowThreshold1);
630             UART_printfLine(2, "02A: %.0fC: %.0fR: %.0f", NextLargestAreaThreshold2, NextLargestColThreshold2, NextLargestRowThreshold2);
631             // UART_printfLine(1, "%.2f %.2f", adcC2Volt, adcC3Volt);
632             // UART_printfLine(2, "%.2f %.2f", adcC4Volt, adcC5Volt);
633         } else if (readbuttons() == 4) {
634             UART_printfLine(1, "03A: %.0fC: %.0fR: %.0f", NextNextLargestAreaThreshold1, NextNextLargestColThreshold1, NextNextLargestRowThreshold1);
635             UART_printfLine(2, "03A: %.0fC: %.0fR: %.0f", NextNextLargestAreaThreshold2, NextNextLargestColThreshold2, NextNextLargestRowThreshold2);
636             // UART_printfLine(1, "L: %.3f R: %.3f", LeftVel, RightVel);
637             // UART_printfLine(2, "uL: %.2f uR: %.2f", uLeft, uRight);
638         } else if (readbuttons() == 8) {
639             UART_printfLine(1, "020x%.2f y%.2f", ladar_pts[20].x, ladar_pts[20].y);
640             UART_printfLine(2, "150x%.2f y%.2f", ladar_pts[150].x, ladar_pts[150].y);
641         } else if (readbuttons() == 3) {
642             UART_printfLine(1, "Vrf: %.2f trn: %.2f", vref, turn);
643             UART_printfLine(2, "MPU: %.2f LPR: %.2f", gyro9250_radians, gyroLPRS10_radians);
644         } else if (readbuttons() == 5) {
645             UART_printfLine(1, "0x: %.2f; 0y: %.2f; 0a: %.2f", OPTITRACKps.x, OPTITRACKps.y, OPTITRACKps.theta);
646             UART_printfLine(2, "State: %d : %d", RobotState, statePos);
647         } else if (readbuttons() == 6) {
648             UART_printfLine(1, "D1 %ld D2 %ld", dis_1, dis_2);
649             UART_printfLine(2, "St1 %ld St2 %ld", measure_status_1, measure_status_2);
650         } else if (readbuttons() == 7) {
651             UART_printfLine(1, "%.0f, %.1f, %.1f, %.1f", tagid, tagx, tagy, tagz);
652             UART_printfLine(2, "%.1f, %.1f, %.1f", tagthetax, tagthetay, tagthetaz);
653         }
654     }
655     UARTPrint = 0;
656 }
657 }
658 }
659
660 __interrupt void SPIB_isr(void) {
661     uint16_t i;
662     GpioDataRegs.GPASET.bit.GPIO32 = 1;
663
664     GpioDataRegs.GPCSET.bit.GPIO66 = 1; // Pull CS high as done R/W
665     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
666
667     if (CurrentChip == MPU9250) {
668         for (i = 0; i < 8; i++) {
669             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
670         }
671
672         PostSWI1(); // Manually cause the interrupt for the SWI1
673     } else if (CurrentChip == DAN28027) {
674         DAN28027Garbage = SpibRegs.SPIRXBUF;
675         dan28027adc1 = SpibRegs.SPIRXBUF;
676         dan28027adc2 = SpibRegs.SPIRXBUF;
677         CurrentChip = MPU9250;
678         SpibRegs.SPIFFCT.bit.TXDLY = 0;
679     }
680
681     SpibRegs.SPIFFRX.bit.RXFFOVFLCLR = 1; // Clear Overflow flag
682     SpibRegs.SPIFFRX.bit.RXFFINTCLR = 1; // Clear Interrupt flag
683     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
684     GpioDataRegs.GPBCLR.bit.GPIO32 = 1;
685 }
686
687 //adcd1 pie interrupt
688 __interrupt void ADCC_ISR (void)
689 {
690     GpioDataRegs.GPASET.bit.GPIO11 = 1;
691     adcc2result = AdccResultRegs.ADCRESULT0;
692     adcc3result = AdccResultRegs.ADCRESULT1;
693     adcc4result = AdccResultRegs.ADCRESULT2;
694     adcc5result = AdccResultRegs.ADCRESULT3;
695
696     // Here covert ADCIND0 to volts
697     adcC2Volt = adcc2result * 3.0 / 4095.0;
698     adcC3Volt = adcc3result * 3.0 / 4095.0;
699     adcC4Volt = adcc4result * 3.0 / 4095.0;
700     adcC5Volt = adcc5result * 3.0 / 4095.0;
701
702     if (MPU9250ignoreCNT >= 1) {
703         CurrentChip = MPU9250;
704     }
705 }

```

```

709     SpibRegs.SPIFFCT.bit.TXDLY = 0;
710     SpibRegs.SPIFFRX.bit.RXFFIL = 0;
711
712     GpioDataRegs.GPCCLEAR.bit.GPI066 = 1;
713
714     SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
715     SpibRegs.SPITXBUF = 0;
716     SpibRegs.SPITXBUF = 0;
717     SpibRegs.SPITXBUF = 0;
718     SpibRegs.SPITXBUF = 0;
719     SpibRegs.SPITXBUF = 0;
720     SpibRegs.SPITXBUF = 0;
721     SpibRegs.SPITXBUF = 0;
722 } else {
723     MPU9250ignoreCNT++;
724 }
725
726 numADCCcalls++;
727 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
728 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
729 GpioDataRegs.GPACLEAR.bit.GPI011 = 1;
730
731 }
732
733 // cpu_timer0_isr - CPU Timer0 ISR
734 __interrupt void cpu_timer0_isr(void)
735 {
736     CpuTimer0.InterruptCount++;
737
738     numTimer0calls++;
739
740     if ((numTimer0calls%5) == 0) {
741         // Blink LaunchPad Red LED
742         //GpioDataRegs.GPBTGGLE.bit.GPI034 = 1;
743     }
744
745     // Acknowledge this interrupt to receive more interrupts from group 1
746     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
747 }
748
749 // cpu_timer1_isr - CPU Timer1 ISR
750 __interrupt void cpu_timer1_isr(void)
751 {
752     serial_sendSCIC(&SerialC, G_command, G_len);
753
754     CpuTimer1.InterruptCount++;
755 }
756
757 // cpu_timer2_isr CPU Timer2 ISR
758 __interrupt void cpu_timer2_isr(void)
759 {
760     // Blink LaunchPad Blue LED
761     //GpioDataRegs.GPATOGGLE.bit.GPI031 = 1;
762     PostSWI3(); //Added PostSWI3 for Lab 7 Ex.1 Step 1 - DS
763     CpuTimer2.InterruptCount++;
764
765     // if ((CpuTimer2.InterruptCount % 10) == 0) {
766     //     UARTPrint = 1;
767     // }
768 }
769
770
771 void setF28027EPWM1A(float controleffort){
772     if (controleffort < -10) {
773         controleffort = -10;
774     }
775     if (controleffort > 10) {
776         controleffort = 10;
777     }
778     float value = (controleffort+10)*3000.0/20.0;
779     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
780 }
781 void setF28027EPWM2A(float controleffort){
782     if (controleffort < -10) {
783         controleffort = -10;
784     }
785     if (controleffort > 10) {
786         controleffort = 10;
787     }
788     float value = (controleffort+10)*3000.0/20.0;
789     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
790 }
791
792 //
793 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
794 //
795 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
796 {
797     GpioDataRegs.GPASET.bit.GPI061 = 1;
798     // Set interrupt priority:
799     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
800     IER |= M_INT12;
801     IER &= MINT12; // Set "global" priority
802     PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
803     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
804     __asm(" NOP");
805     EINT;
806
807     //#####
808     IMU_data[0] = readdata[1];
809     IMU_data[1] = readdata[2];
810     IMU_data[2] = readdata[3];
811     IMU_data[3] = readdata[5];
812     IMU_data[4] = readdata[6];
813     IMU_data[5] = readdata[7];
814
815     accelx = (((float)(IMU_data[0]))*4.0/32767.0);
816     accely = (((float)(IMU_data[1]))*4.0/32767.0);
817     accelz = (((float)(IMU_data[2]))*4.0/32767.0);
818     gyrox = (((float)(IMU_data[3]))*250.0/32767.0);
819     gyroy = (((float)(IMU_data[4]))*250.0/32767.0);
820     gyroz = (((float)(IMU_data[5]))*250.0/32767.0);
821
822     if(calibration_state == 0){
823         calibration_count++;
824         if (calibration_count == 2000) {
825             calibration_state = 1;
826             calibration_count = 0;
827         }
828     } else if(calibration_state == 1){
829         accelx_offset+=accelx;
830         accely_offset+=accely;
831         accelz_offset+=accelz;
832         gyrox_offset+=gyrox;
833         gyroy_offset+=gyroy;
834         gyroz_offset+=gyroz;
835         gyroLPRS10_offset+=adcC5Volt;
836         calibration_count++;
837         if (calibration_count == 2000) {
838             calibration_state = 2;
839             accelx_offset/=2000.0;
840             accely_offset/=2000.0;
841             accelz_offset/=2000.0;
842             gyrox_offset/=2000.0;
843             gyroy_offset/=2000.0;
844             gyroz_offset/=2000.0;
845             gyroLPRS10_offset/=2000.0;
846             calibration_count = 0;
847         }
848     } else if(calibration_state == 2) {
849         gyroLPRS10_drift += (((adcC5Volt-gyroLPRS10_offset) + old_gyroLPRS10)*.0005)/(2000);
850

```

```

851     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
852
853     gyro9250_drift += (((gyroz-gyroz_offset) + old_gyro9250)*.0005)/(2000);
854     old_gyro9250 = gyro9250_offset;
855     calibration_count++;
856
857     if (calibration_count == 2000) {
858         calibration_state = 3;
859         calibration_count = 0;
860         doneCal = 1;
861         newOPTITRACKpose = 0;
862     }
863 } else if(calibration_state == 3){
864     accelx -=(accelx_offset);
865     accely -=(accely_offset);
866     accelz -=(accelz_offset);
867     gyrox -= gyrox_offset;
868     gyroy -= gyroy_offset;
869     gyroz -= gyroz_offset;
870     LeftWheel = readEncLeft();
871     RightWheel = readEncRight();
872     HandValue = readEncWheel();
873
874     gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
875     old_gyro9250 = gyroz;
876
877     gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
878     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
879
880     gyro9250_radians = (gyro9250_angle * (PI/180.0));
881     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
882
883     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
884     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
885
886     if (newLinuxCommands == 1) {
887         newLinuxCommands = 0;
888         printLinux1 = LinuxCommands[0];
889         printLinux2 = LinuxCommands[1];
890         ///? = LinuxCommands[2];
891         ///? = LinuxCommands[3];
892         ///? = LinuxCommands[4];
893         ///? = LinuxCommands[5];
894         ///? = LinuxCommands[6];
895         ///? = LinuxCommands[7];
896         ///? = LinuxCommands[8];
897         ///? = LinuxCommands[9];
898         ///? = LinuxCommands[10];
899     }
900
901     if (NewCAMDataThreshold1 == 1) {
902         NewCAMDataThreshold1 = 0;
903         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
904         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
905         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
906
907         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
908         robotToBall1 = -0.00092154*MaxRowThreshold1*MaxRowThreshold1*MaxRowThreshold1 + 0.006486*MaxRowThreshold1*MaxRowThreshold1 - 0.6728*MaxRowThreshold1 + 25.42;
909
910         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
911         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
912         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
913
914         NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
915         NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
916         NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
917         numThres1++;
918         if ((numThres1 % 5) == 0) {
919             // LED4 is GPIO97
920             GpioDataRegs.GPDTOGGLE.bit.GPIO97 = 1;
921         }
922     }
923
924     if (NewCAMDataThreshold2 == 1) {
925         NewCAMDataThreshold2 = 0;
926         MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
927         MaxColThreshold2 = fromCAMvaluesThreshold2[1];
928         MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
929         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
930         robotToBall2 = -0.00092154*MaxRowThreshold2*MaxRowThreshold2*MaxRowThreshold2 + 0.006486*MaxRowThreshold2*MaxRowThreshold2 - 0.6728*MaxRowThreshold2 + 25.42;
931
932         NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
933         NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
934         NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
935
936         NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
937         NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
938         NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
939         numThres2++;
940         if ((numThres2 % 5) == 0) {
941             // LED5 is GPIO111
942             GpioDataRegs.GPDTOGGLE.bit.GPIO111 = 1;
943         }
944     }
945
946     if (NewCAMDataAprilTag1 == 1) {
947         NewCAMDataAprilTag1 = 0;
948         tagx = fromCAMvaluesAprilTag1[0];
949         tagy = fromCAMvaluesAprilTag1[1];
950         tagz = fromCAMvaluesAprilTag1[2];
951
952         tagthetax = fromCAMvaluesAprilTag1[3];
953         tagthetay = fromCAMvaluesAprilTag1[4];
954         tagthetaz = fromCAMvaluesAprilTag1[5];
955
956         tagid = fromCAMvaluesAprilTag1[6];
957
958         numtag++;
959         if ((numtag % 5) == 0) {
960             // LED2 is GPIO94
961             GpioDataRegs.GPCTOGGLE.bit.GPIO94 = 1;
962         }
963     }
964
965     if (NewLVData == 1) {
966         NewLVData = 0;
967         printLV1 = fromLVvalues[0];
968         printLV2 = fromLVvalues[1];
969         ///? = fromLVvalues[2];
970         ///? = fromLVvalues[3];
971         ///? = fromLVvalues[4];
972         ///? = fromLVvalues[5];
973         ///? = fromLVvalues[6];
974         ///? = fromLVvalues[7];
975     }
976
977
978     if (new_optitrack == 1) {
979         OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
980         new_optitrack = 0;
981     }
982
983     // Step 0: update B, u
984     B[0][0] = cosf(ROBOTps.theta)*0.001;
985     B[1][0] = sinf(ROBOTps.theta)*0.001;
986     B[2][1] = 0.001;
987     u[0][0] = 0.5*(LeftVel + RightVel); // linear velocity of robot
988     u[1][0] = gyroz*(PI/180.0); // angular velocity in rad/s (negative for right hand angle)
989
990     // Step 1: predict the state and estimate covariance
991     Matrix3x2_Mult(B, u, Bu); // Bu = B*u
992     Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu

```



```

993 Matrix3x3_Add(P_pred, 0, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
994 // Step 2: if there is a new measurement, then update the state
995 if (1 == newOPTITRACKpose) {
996     OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
997     if ((OptiNumUpdatesProcessed%10)==0) {
998         // LED 3
999         GpioDataRegs.GPCTOGGLE.bit.GPIO95 = 1;
1000     }
1001     newOPTITRACKpose = 0;
1002     z[0][0] = OPTITRACKps.x; // take in the Optitrack Motion Capture measurement
1003     z[1][0] = OPTITRACKps.y;
1004     // fix for OptiTrack problem at 180 degrees
1005     if (cosf(ROBOTps.theta) < -0.99) {
1006         z[2][0] = ROBOTps.theta;
1007     }
1008     else {
1009         z[2][0] = OPTITRACKps.theta;
1010     }
1011     // Step 2a: calculate the innovation/measurement residual, ytilde
1012     Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0); // ytilde = z-x_pred
1013     // Step 2b: calculate innovation covariance, S
1014     Matrix3x3_Add(P_pred, R, S, 1.0, 1.0); // S = P_pred + R
1015     // Step 2c: calculate the optimal Kalman gain, K
1016     Matrix3x3_Invert(S, S_inv);
1017     Matrix3x3_Mult(P_pred, S_inv, K); // K = P_pred*(S^-1)
1018     // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
1019     Matrix3x1_Mult(K, ytilde, temp_3x1);
1020     Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
1021     // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
1022     Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
1023     Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
1024 } // end of correction step
1025
1026 // set ROBOTps to the updated and corrected Kalman values.
1027 ROBOTps.x = x_pred[0][0];
1028 ROBOTps.y = x_pred[1][0];
1029 ROBOTps.theta = x_pred[2][0];
1030
1031 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1032 // uses xy code to step through an array of positions
1033 if (xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1034     statePos = (statePos+1)%NUMWAYPOINTS;
1035 }
1036 // state machine
1037 colcentroid1 = MaxColThreshold1 - 80;
1038 colcentroid2 = MaxColThreshold2 - 80; // 80 is used as the center column of the camera, so a centered green or orange ball centered gives centroid = 0 - DS
1039 switch (RobotState) {
1040 case 1: // command the robot to an X,Y point in the course - KL
1041     // vref and turn are the vref and turn returned from xy_control
1042     if (LADARfront < 1.2) {
1043         vref = 0.2;
1044         checkfronttally++;
1045         if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
1046             RobotState = 10; // Wall follow
1047             WallFollowtime = 0;
1048             right_wall_follow_state = 1;
1049         }
1050     } else {
1051         checkfronttally = 0;
1052     }
1053     // ensures that the robot does not move for 2 seconds after returning to state 1, so after is stops detecting a golf ball - ER
1054     statecount++;
1055     if (statecount<=2000){
1056         RobotState = 1;
1057     }else{
1058         if (MaxAreaThreshold1 > 30) { // green ball detection threshold chosen so camera noise (image disturbance, not literal) does not trigger chasing rolling ball behavior - DS
1059             RobotState = 20;
1060             statecount = 0;
1061         }
1062         if (MaxAreaThreshold2 > 30) { // same detection threshold but for orange ball - DS
1063             RobotState = 30;
1064             statecount = 0;
1065         }
1066     }
1067     break;
1068 case 10: // right wall following if an obstacle gets in the way of robot - KL
1069     if (right_wall_follow_state == 1) {
1070         //Left turn
1071         turn = Kp_front_wall*(14.5 - LADARfront);
1072         vref = front_turn_velocity;
1073         if (LADARfront > left_turn_Stop_threshold) {
1074             right_wall_follow_state = 2;
1075         }
1076     } else if (right_wall_follow_state == 2) {
1077         //Right Wall Follow
1078         turn = Kp_right_wall*(ref_right_wall - LADARrightfront);
1079         vref = foward_velocity;
1080         if (LADARfront < left_turn_Start_threshold) {
1081             right_wall_follow_state = 1;
1082         }
1083     }
1084     if (turn > turn_saturation) {
1085         turn = turn_saturation;
1086     }
1087     if (turn < -turn_saturation) {
1088         turn = -turn_saturation;
1089     }
1090     WallFollowtime++;
1091     // exit wall following if at least 5 seconds have passed and front path is clear - KL
1092     if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1093         RobotState = 1; //return to XY waypoint navigation - KL
1094         checkfronttally = 0;
1095     }
1096     break;
1097 // this state sets vref and turn so that the robot moves towards a detected green golf ball and after 1 second of waiting, sends the robot to state 22 which stops the robot - ER
1098 case 20:
1099     // put vision code here
1100     if (MaxColThreshold1 == 0 || MaxAreaThreshold1 < 3){
1101         vref = 0;
1102         turn = 0;
1103     } else {
1104         vref = 0.75;
1105         turn = kpvision * (0 - colcentroid1);
1106         if (MaxRowThreshold1 > 100){ //KLEC: row number for rb to know how to close it is to the ball, need to tune the number for enough clearance so that gate doesn't push ball away
1107             RobotState = 22;
1108             count = 0;
1109         }
1110     }
1111     break;
1112 case 22: // after no longer detecting the green golf ball, the robot stops moving - ER, lift arm and move tongue
1113     vref = 0;
1114     turn = 0;
1115     count++;
1116     if (count>=1000){//after 1 second, robot drives forward - KL
1117         RobotState = 24;
1118         count = 0;
1119     }
1120     break;
1121 case 24: // robot drives forward into the golf ball - KL
1122     vref = 0.5;
1123     turn = 0;
1124     setEPWM5B_RCServo(LTong); //KLEC: move the tongue to the left side, green balls go to the right compartment
1125     setEPWM6A_RCServo(Gate_0); //KLEC: open the gate
1126     count++;
1127     if (count>=2000){
1128         RobotState = 26;
1129         count = 0;
1130     }
1131     break;
1132 case 26: // robot pauses once ball is presumed to be captured - KL

```



```

1135     vref = 0;
1136     turn = 0;
1137     setEPWM6A_RCServo(Gate_C);
1138     setEPWM5B_RCServo(MTong);
1139     count++;
1140     if (count>=1000){ //after 1 second, robot returns to waypoint navigation - KL
1141         RobotState = 1;
1142         count = 0;
1143         statecount = 0;
1144     }
1145     break;
1146
1147     // states for orange ball
1148 case 30:
1149     // put vision code here
1150     //saying that if the orange ball is not detected or doesn't fit in the area threshold, then to stop the robot - KP
1151     //this helps it not detect other orange objects in the room as the ball - KP
1152     if (MaxColThreshold2 == 0 || MaxAreaThreshold2 < 3){
1153         vref = 0;
1154         turn = 0;
1155     } else {
1156         //tells it what to do if it is detected, move to state 32 - KP
1157         vref = 0.75;
1158         turn = kpvision * (0 - colcentroid2);
1159         if (MaxRowThreshold2 > 100){
1160             RobotState = 32;
1161             count = 0;
1162         }
1163     }
1164     break;
1165 case 32:
1166     //stopping and pausing with vref and turn at 0 - KP
1167     vref = 0;
1168     turn = 0;
1169     setEPWM5B_RCServo(RTong); //KLEC: move the tongue to the right side, orange balls go to the left compartment
1170     setEPWM6A_RCServo(Gate_0); //KLEC: open the gate
1171     count++;
1172     if (count>=1000){
1173         RobotState = 34;
1174         count = 0;
1175     }
1176     break;
1177 case 34:
1178     //telling robot to move forward then move to state 36 - KP
1179     vref = 0.5;
1180     turn = 0;
1181     setEPWM5B_RCServo(RTong);
1182     setEPWM6A_RCServo(Gate_0);
1183     count++;
1184     if (count>=1000){
1185         RobotState = 36;
1186         count = 0;
1187     }
1188     break;
1189 case 36:
1190     //another stop before it begins the route again - KP
1191     vref = 0;
1192     turn = 0;
1193     setEPWM6A_RCServo(Gate_C);
1194     setEPWM5B_RCServo(MTong);
1195     count++;
1196     if (count>=1000){
1197         RobotState = 1;
1198         count = 0;
1199         statecount = 0;
1200     }
1201     break;
1202
1203 default:
1204     break;
1205 }
1206
1207
1208 //Must be called each time into this SWI1 function
1209 PIControl(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1210 // These below lines also must be called each time into this SWI1 function
1211 setEPWM1A(uLeft);
1212 setEPWM2A(-uRight);
1213 setF28027EPWM1A(uLeft);
1214 setF28027EPWM2A(-uRight);
1215 LeftWheel_1 = LeftWheel;
1216 RightWheel_1 = RightWheel;
1217
1218 if((timecount%250) == 0) {
1219     DataToLabView.floatData[0] = ROBOTps.x;
1220     DataToLabView.floatData[1] = ROBOTps.y;
1221     DataToLabView.floatData[2] = ROBOTps.theta;
1222     DataToLabView.floatData[3] = (float)timecount;
1223     DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1224     DataToLabView.floatData[5] = (float)RobotState;
1225     DataToLabView.floatData[6] = (float)statePos;
1226     DataToLabView.floatData[7] = LADARfront;
1227     LVsenddata[0] = '+'; // header for LVdata
1228     LVsenddata[1] = '$';
1229     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1230         if (i%2==0) {
1231             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1232         } else {
1233             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1234         }
1235     }
1236     serial_sendSCID(&SerialD, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1237 }
1238
1239 }
1240 timecount++;
1241 if((timecount%200) == 0)
1242 {
1243     if(doneCal == 0) {
1244         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1245     }
1246     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1247     UARTPrint = 1; // Tell While loop to print
1248 }
1249
1250 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1251 CurrentChip = DAN28027;
1252 SpibRegs.SPIFFRX.bit.RXFIL = 3;
1253 GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
1254 SpibRegs.SPITXBUF = 0x000A;
1255 SpibRegs.SPITXBUF = EPwm1A_F28027;
1256 SpibRegs.SPITXBUF = EPwm2A_F28027;
1257
1258 //*****
1259 //
1260 // Restore registers saved:
1261 //
1262 DINT;
1263 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1264
1265 GpioDataRegs.GPB CLEAR.bit.GPIO61 = 1;
1266 }
1267
1268 //
1269 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1270 //
1271 __interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1272 {
1273     // Set interrupt priority:
1274     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1275     IER |= M_INT12;
1276     IER &= MINT12; // Set "global" priority

```

```

1277 PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1278 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1279 __asm(" NOP");
1280 EINT;
1281
1282 //*****
1283 // Insert SWI ISR Code here.....
1284 // LED1
1285 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1286 if (LADARpingpong == 1) {
1287 // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1288 LADARrightfront = 19; // 19 is greater than max feet
1289 for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1290 if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1291 LADARrightfront = ladar_data[LADARi].distance_ping;
1292 }
1293 }
1294 // LADARfront is the min of dist 111, 112, 113, 114, 115
1295 LADARfront = 19;
1296 for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1297 if (ladar_data[LADARi].distance_ping < LADARfront) {
1298 LADARfront = ladar_data[LADARi].distance_ping;
1299 }
1300 }
1301 LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1302 LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1303 for (LADARi = 0; LADARi < 228; LADARi++) {
1304
1305 ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1306 ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1307
1308 }
1309 } else if (LADARpingpong == 0) {
1310 // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1311 LADARrightfront = 19; // 19 is greater than max feet
1312 for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1313 if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1314 LADARrightfront = ladar_data[LADARi].distance_pong;
1315 }
1316 }
1317 // LADARfront is the min of dist 111, 112, 113, 114, 115
1318 LADARfront = 19;
1319 for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1320 if (ladar_data[LADARi].distance_pong < LADARfront) {
1321 LADARfront = ladar_data[LADARi].distance_pong;
1322 }
1323 }
1324 LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1325 LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1326 for (LADARi = 0; LADARi < 228; LADARi++) {
1327
1328 ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1329 ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1330
1331 }
1332 }
1333
1334
1335 //*****
1336 //
1337 // Restore registers saved:
1338 //
1339 DINT;
1340 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1341
1342 }
1343
1344 //
1345 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1346 //
1347 __interrupt void SWI3_LowestPriority(void) // FLASH_CORRECTABLE_ERROR
1348 {
1349 // Set interrupt priority:
1350 volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1351 IER |= MINT12;
1352 IER &= MINT12; // Set "global" priority
1353 PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1354 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1355 __asm(" NOP");
1356 EINT;
1357 RCangle = readEncWheel(); //Lab RC servo exercise uses encoder angle as the command input for servo motion - DS
1358 // setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
1359 // setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
1360 // setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use
1361 setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Tongue
1362 setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Gate
1363 //*****
1364 // Insert SWI ISR Code here.....
1365
1366 //*****
1367 //
1368 // Restore registers saved:
1369 //
1370 DINT;
1371 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1372
1373 }
1374
1375 // ----- code for CAN start here -----
1376 __interrupt void can_isr(void)
1377 {
1378 int i = 0;
1379
1380 uint32_t status;
1381
1382 //
1383 // Read the CAN interrupt status to find the cause of the interrupt
1384 //
1385 status = CANGetInterruptCause(CANB_BASE);
1386
1387 //
1388 // If the cause is a controller status interrupt, then get the status
1389 //
1390 if(status == CAN_INT_INT0ID_STATUS)
1391 {
1392 //
1393 // Read the controller status. This will return a field of status
1394 // error bits that can indicate various errors. Error processing
1395 // is not done in this example for simplicity. Refer to the
1396 // API documentation for details about the error status bits.
1397 // The act of reading this status will clear the interrupt.
1398 //
1399 status = CANGetStatus(CANB_BASE);
1400
1401 }
1402
1403 //
1404 // Check if the cause is the receive message object 2
1405 //
1406 else if(status == RX_MSG_OBJ_ID_1)
1407 {
1408 //
1409 // Get the received message
1410 //
1411 CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1412
1413 for(i = 0; i<2; i++)
1414 {
1415 dis_raw_1[i] = rxMsgData[i];
1416 }
1417
1418 dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];

```

```

1419     measure_status_1 = rxMsgData[2];
1420
1421     //
1422     // Getting to this point means that the RX interrupt occurred on
1423     // message object 2, and the message RX is complete. Clear the
1424     // message object interrupt.
1425     //
1426     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1427
1428     //
1429     // Increment a counter to keep track of how many messages have been
1430     // received. In a real application this could be used to set flags to
1431     // indicate when a message is received.
1432     //
1433     rxMsgCount_1++;
1434
1435     //
1436     // Since the message was received, clear any error flags.
1437     //
1438     errorFlag = 0;
1439 }
1440
1441 else if(status == RX_MSG_OBJ_ID_2)
1442 {
1443     //
1444     // Get the received message
1445     //
1446     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1447
1448     for(i = 0; i<2; i++)
1449     {
1450         dis_raw_2[i] = rxMsgData[i];
1451     }
1452
1453     dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1454
1455     measure_status_2 = rxMsgData[2];
1456
1457     //
1458     // Getting to this point means that the RX interrupt occurred on
1459     // message object 2, and the message RX is complete. Clear the
1460     // message object interrupt.
1461     //
1462     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1463
1464     //
1465     // Increment a counter to keep track of how many messages have been
1466     // received. In a real application this could be used to set flags to
1467     // indicate when a message is received.
1468     //
1469     rxMsgCount_2++;
1470
1471     //
1472     // Since the message was received, clear any error flags.
1473     //
1474     errorFlag = 0;
1475 }
1476
1477 else if(status == RX_MSG_OBJ_ID_3)
1478 {
1479     //
1480     // Get the received message
1481     //
1482     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1483
1484     for(i = 0; i<4; i++)
1485     {
1486         lightlevel_raw_1[i] = rxMsgData[i];
1487         quality_raw_1[i] = rxMsgData[i+4];
1488     }
1489
1490     lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1491     quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1492
1493     //
1494     // Getting to this point means that the RX interrupt occurred on
1495     // message object 2, and the message RX is complete. Clear the
1496     // message object interrupt.
1497     //
1498     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1499
1500     //
1501     // Since the message was received, clear any error flags.
1502     //
1503     errorFlag = 0;
1504 }
1505
1506 else if(status == RX_MSG_OBJ_ID_4)
1507 {
1508     //
1509     // Get the received message
1510     //
1511     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1512
1513     for(i = 0; i<4; i++)
1514     {
1515         lightlevel_raw_2[i] = rxMsgData[i];
1516         quality_raw_2[i] = rxMsgData[i+4];
1517     }
1518
1519     lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1520     quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1521
1522     //
1523     // Getting to this point means that the RX interrupt occurred on
1524     // message object 2, and the message RX is complete. Clear the
1525     // message object interrupt.
1526     //
1527     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1528
1529     //
1530     // Since the message was received, clear any error flags.
1531     //
1532     errorFlag = 0;
1533 }
1534
1535 //
1536 // If something unexpected caused the interrupt, this would handle it.
1537 //
1538 else
1539 {
1540     //
1541     // Spurious interrupt handling can go here.
1542     //
1543 }
1544
1545 //
1546 // Clear the global interrupt flag for the CAN interrupt line
1547 //
1548 CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1549
1550 //
1551 // Acknowledge this interrupt located in group 9
1552 //
1553 InterruptClearACKGroup(INTERRUPT_ACK_GROUP9);

```

```
1561 }  
1562 // ----- code for CAN end here -----
```